PYTHON MID-TERM PROJECT REQUIREMENTS

Fall 2026–27 Semester | CSC 4162. Programming in Python

Group Project | Total: 40 Marks

ITEM	REQUIREMENT
Assessment type	Group project.
Scope	Python fundamentals, control flow, functions, data structures, file handling, exception handling, object-oriented programming (OOP), and introductory library use.
Main output	A working Python application with user interaction, persistent data, structured logic, and basic calculations or analysis.
Assessment principle	Marks are based primarily on correctness, code structure, data handling, useful functionality, and the ability to explain the implementation—not on visual design alone.

ASSESSMENT INTENT

Develop a small, complete application that addresses a practical problem. A simple interface with reliable logic and well-structured code is preferable to an attractive interface with weak implementation. Choose a focused idea that can be completed well and may be extended later with additional data-driven functionality for the final term.

1. Project Objective

Develop a Python application that accepts user input, processes information, stores or retrieves records from files, handles common errors, and presents meaningful output.

The implementation should demonstrate:

- Variables and data types
- Operators and conditional statements
- Loops and functions
- Lists, tuples, sets, and dictionaries
- File handling and exception handling
- Object-oriented programming

Use at least one approved library or interface module for a meaningful feature. NumPy may support numerical calculations, Pygame may support interactive gameplay, and Tkinter may provide a graphical interface. Other libraries require instructor approval.

2. Prospective Project Types

PROJECT TYPE	EXPECTED USE OF LIBRARY	EXAMPLE IDEAS
Data analysis application	Use NumPy to calculate summaries from records or numerical values.	Expense tracker, attendance analyzer, student result analyzer, community resource tracker.
GUI management application	Use Tkinter for input forms, buttons, tables, and messages.	Library borrowing system, food donation coordinator, waste collection tracker.
Game or interactive application	Use Pygame for events, keyboard/mouse input, scoring, timing, and feedback.	Educational quiz, road-safety quiz, recycling challenge, typing speed tester.
Hybrid application	Combine an interface with file-based records and useful calculations.	Health habit tracker, local issue reporting prototype, inventory and sales tracker.

3. Suggested Project Domains: Societal Impact

Select a domain where a small application could help people organize information, access a service, or make a practical decision.

The ideas below are starting points, not mandatory titles. Identify a specific user group and a manageable problem.

DOMAIN	POSSIBLE APPLICATION IDEAS	POTENTIAL USERS OR BENEFIT
Education and learning	Study planner, learning-resource organizer, quiz practice system, attendance or result manager.	Students and teachers; support learning organization and progress tracking.
Health and well-being	Appointment organizer, medicine reminder log, healthy-habit tracker, community fitness tracker.	Individuals or community groups; support routine tracking. Not a diagnostic tool.
Environment and sustainability	Waste-sorting guide, recycling collection tracker, household energy-use log, community clean-up organizer.	Households, students, and local groups; encourage sustainable practices.
Food security and community support	Food donation coordinator, community pantry inventory, surplus-food record system.	Volunteers and community organizations; help coordinate resources.
Personal finance and consumer awareness	Budget planner, expense tracker, savings-goal manager, price-comparison record system.	Individuals and households; improve spending visibility.
Accessibility and inclusion	Accessible service directory, caregiver task organizer, categorized resource finder.	People with access needs, caregivers, and community support groups.
Public services and civic engagement	Local issue-reporting prototype, public facility directory, community request tracker.	Residents and local organizations; organize reports and follow-up.
Transport and road safety	Commute expense tracker, road-safety quiz, public transport feedback log, cycling activity tracker.	Commuters and students; support awareness and record keeping.
Agriculture and local livelihoods	Crop activity log, farm inventory tracker, market-price record book, irrigation schedule manager.	Small farmers and local vendors; organize operational records.
Small business and livelihoods	Inventory and sales tracker, order manager, service booking system, local vendor directory.	Small businesses and customers; improve record keeping and service coordination.

RESPONSIBLE USE
Avoid collecting unnecessary personal or sensitive information. Use fictional or non-sensitive sample records where possible. Explain the specific user need and intended benefit; a socially relevant topic alone does not demonstrate impact.

4. Required Python Concepts

CONCEPT	EXPECTED EVIDENCE
Variables and data types	Store names, IDs, values, categories, choices, statuses, and calculated results.
Operators	Perform calculations, comparisons, totals, averages, percentages, and validation checks.
Conditional statements	Use `if`, `elif`, and `else` for choices, validation, status decisions, and messages.
Loops	Repeat menus, process records, traverse collections, or run interactive rounds.
Functions	Separate input, validation, calculation, search, update, delete, save/load, and summary tasks.
List	Store multiple records, items, transactions, scores, or objects.
Tuple	Represent fixed values such as categories, menu labels, options, or configuration values.
Set	Track unique IDs or categories and help prevent duplicates where appropriate.
Dictionary	Represent records as key-value pairs or map IDs and categories to related information.
File handling	Save and load data using `.txt`, `.csv`, or `.json`. Data must persist after the program closes.
Exception handling	Handle invalid input, missing files, empty or malformed data, and unavailable records.
Object-oriented programming	Use at least one meaningful class representing an entity such as a Student, Product, Expense, Book, Report, or Activity.
Approved library/module	Use NumPy, Pygame, Tkinter, or another instructor-approved module for a genuine feature.

5. Minimum Functional Requirements

ID	REQUIREMENT	EXPECTATION
FR-1	User interaction	Users can enter information, select actions, and view results through a GUI, visual interface, or clear menu-driven interaction.
FR-2	At least five meaningful operations	Examples include add, view, search, update, delete, calculate summaries, save/load, play a round, or show a leaderboard.
FR-3	Persistent data	Save and load records or results. Data must not disappear when the program closes.
FR-4	Validation	Handle invalid input such as negative values, empty required fields, duplicate IDs, or invalid choices.
FR-5	Analysis or calculation	Generate useful output such as totals, averages, status, category summaries, score statistics, or inventory value.
FR-6	Structured implementation	Use functions and classes; do not put all application logic in one large block.
FR-7	Error recovery	Continue after ordinary user mistakes rather than crashing.

6. Approved Library or Interface Options

OPTION	EXPECTED USE
NumPy	Arrays and numerical summaries such as totals, averages, minimum/maximum, median, or standard deviation.
Tkinter	GUI elements such as labels, entry fields, buttons, list boxes, tables, and message boxes.
Pygame	Visual interaction, event handling, keyboard/mouse input, scoring, timing, and feedback.
Other library	Requires instructor approval. It must support a real feature and must not replace students' implementation of core Python logic.

IMPORTANT
Importing a library is not sufficient. Demonstrate how it contributes to the application.

7. Expected Program Structure

COMPONENT	PURPOSE
Data model class	Represents a main entity such as a student, product, expense, book, user, or activity.
Manager/application class	Coordinates records, operations, file handling, and summaries.
Input and validation functions	Collect information and check its correctness.
Calculation functions	Compute totals, averages, status, scores, or category summaries.
File functions	Save and load records with appropriate exception handling.
Interface functions	Display menus, forms, buttons, screens, results, and messages.

8. File Handling and Data Requirements

- Use `.txt`, `.csv`, or `.json` files to save and load application records or results.
- Load existing data when appropriate, or provide a clear load operation.
- Handle missing, empty, or incorrectly formatted files gracefully.
- Include all sample data files required to run the application.
- For record-management applications, at least 10 meaningful sample records are recommended. Interactive projects should contain enough questions or result entries to demonstrate functionality.

9. Exception Handling Requirements

CASE	REQUIRED HANDLING
Invalid menu or action	Show a clear message and allow the user to continue.
Non-numeric input	Use `try-except` and ask again or show an understandable error.
Out-of-range value	Reject impossible or invalid values appropriate to the application.
Duplicate identifier	Use a set or suitable search logic to prevent duplicates where required.
Record not found	Handle search, update, and delete requests for missing records gracefully.
File error	Do not terminate unexpectedly because a file is missing or unreadable.

10. Restrictions

- The project must demonstrate Python fundamentals; libraries must not hide the required programming logic.
- Standard-library modules such as `csv`, `json`, `os`, `random`, `math`, and `datetime` may be used when relevant.
- Do not submit a copied project or code that group members cannot explain. Follow the course's academic integrity policy.
- Keep the scope manageable. A complete, well-structured application is preferable to an ambitious but unfinished system.

11. Mark Distribution

ASSESSMENT CRITERION	EVIDENCE EXPECTED	MARKS
Problem overview and project design	Clear problem statement, objectives, selected application type, and intended users.	4
User interaction and interface	Meaningful input and output through a menu, GUI, or visual interface.	5
Variables, operators, conditionals, and loops	Correct data use, calculations, decisions, repetition, and program flow.	4
Functions and modularity	At least five meaningful functions/methods and clear separation of tasks.	4
Data structures	Meaningful use of list, tuple, set, and dictionary.	5
File handling	Working save/load, persistent data, and readable file organization.	4
Exception handling and validation	Robust handling of invalid input, missing records, and file errors.	3
Object-oriented programming	At least one meaningful class with clear attributes and methods.	5
Library/module use and analysis	Purposeful library use and meaningful analysis or interaction.	4
Code quality, report, and presentation	Readable code, appropriate comments, complete report, clear demonstration, and explanation.	2
	TOTAL	**40**

12. Submission Requirements

ITEM	DETAILS
Source code	Submit all `.py` files. Clearly identify the main file, such as `main.py` or `project.py`. Use of GitHub/GitLab is encouraged.
Data files	Submit all required sample `.txt`, `.csv`, or `.json` files.
Project report	Explain the problem, design, features, Python concepts, data structures, OOP, file handling, exception handling, and library/module use.
Screenshots	Include screenshots of major features, interface/menu/game screens, saved output, and summaries/results.
Demonstration	Provide a short live demonstration and be prepared for viva questions.

13. Required Report Format

1. Project title and group/member information.
2. Problem statement, objectives, and intended users.
3. Feature list and application workflow/design.
4. Explanation of the Python concepts used.
5. Data structure explanation: list, tuple, set, and dictionary.
6. Object-oriented design: class names, attributes, and methods.
7. File handling and exception handling explanation.
8. Library/module use and justification.
9. Screenshots and sample input/output.
10. Limitations, societal benefit, and possible future improvements.

14. Demonstration and Viva Expectations

DEMONSTRATION ITEM	EXPECTATION
Run the application	Show that the submitted code runs correctly.
Show main features	Demonstrate the main operations relevant to the project.
Show file save/load	Prove that data persists after saving and loading.
Explain code	Explain important functions, classes, data structures, validation, and exception handling.
Answer concept questions	Be prepared to discuss Python fundamentals and the selected library/module.

15. Academic Integrity

Students may consult documentation, course materials, and small examples, but the submitted project must be their own work.

Submitting a copied project, making only superficial changes to an online project, or submitting code that group members cannot explain may result in penalties according to course policy.

During evaluation, students may be asked to explain or modify a function, change a condition, add a small feature, or trace the output.

16. Final Instructions

Build a small, correct, complete application.
Choose a manageable problem with a clear user need. Implement the required Python concepts meaningfully, handle data and errors carefully, and explain your design decisions.
Consider how the application might be extended in the future with additional data or analytical functionality, but focus this project on a reliable prototype.